

Principios del *testing* ágil

Metodologías ágiles

Antes todo deberíamos recordar algunos puntos de las metodologías ágiles:

Manifiesto ágil

Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:

- Individuos e interacciones sobre procesos y herramientas.
- Software funcionando sobre documentación extensiva.
- Colaboración con el cliente sobre negociación contractual.
- Respuesta ante el cambio sobre seguir un plan.

Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.

¿Qué significa calidad en agilidad?

Agilidad es calidad

La calidad es satisfacer las expectativas de las partes interesada, y para ello no hay nada mejor que priorizar los objetivos en función de los que aportan más valor al negocio.

La calidad es que el producto y servicio puedan crecer de manera sostenible, que estos cambios tengan un impacto controlado, de manera que estos cambios sean económicos.

Debemos utilizar prácticas de ingeniería de software (patrones de diseño, revisión por pares, etc.) y cuidamos la calidad interna del producto empezando primero por las pruebas (TDD, *refactoring*).

Mientras construimos el producto, comprobamos de manera continua que se comportan de manera adecuada. Comprobando que sean simples automatización de integraciones y pruebas.

Las metodologías ágiles no se olvidan del capital humano que lo hace posible. La calidad depende de las personas que trabajan en él y de cómo colaboran, mucho más que de tener procesos bien definidos y documentados.

¿Qué se necesitó para ser ágil?

- Un cambio de cultura en individuos, organizaciones, y especialmente en alta gerencia.
- Crear equipos más productivos y motivados basados en la colaboración entre sus miembros.
- Realizar *coaching* para que el equipo adquiriera conocimiento de forma colaborativa.
- Flexibilidad para realizar cambios.
- Enfocarnos en proporcionar valor en todas las tareas que hacemos.

Entonces, la agilidad es una gran oportunidad para tener más competitivas y oportunidad para ser más efectivos y eficaces.

Los métodos ágiles se basan en los siguientes conceptos básicos:

- Basados en el cambio.
- Ciclos rápidos, despliegue frecuente.
- Diseño Simple (YAGNI: *You Aren't Going to Need It*).
- *Refactoring*.
- Programación por pares.
- Retrospectiva.
- Conocimiento tácito.
- Desarrollo basado en test (*Test driven*).

Testing ágil (pruebas ágiles)

Tratar el desarrollo como el cliente de pruebas, haciendo hincapié en el paradigma de diseño de prueba en primer lugar.

Las pruebas son obtenidas por el equipo, ya que se está analizando y diseñando las pruebas del sistema al momento del desarrollo.

¿Qué son pruebas ágiles?

Dentro de un equipo ágil existe comunicación continua entre los integrantes, definiendo los requisitos, las pruebas y el comportamiento esperado del producto. Además, se manifiesta el concepto de roles, donde se intenta que cada miembro posea un buen número de roles para eliminar la dependencia entre individuos, promoviendo siempre a todos los principios de calidad en sus actividades diarias. En este marco, en un equipo ágil todos los individuos tienen habilidades de probadores.

Un probador ágil se caracteriza por tener, además de habilidades técnicas en *testing*, destreza para integrar las necesidades del interesado al equipo desarrollador, definiendo las historias del usuario en requisitos y, también, transformándolas en parámetros de prueba cuando se codifiquen. El rol de probador sobresale por colaborar con todos los integrantes del equipo.

Los conceptos y definiciones que giran alrededor de la prueba ágil son:

Tester

Persona del equipo que puede desempeñar varios roles en un proyecto de software. Tiene habilidades y destrezas para el desarrollo y ejecución de pruebas.

Interesado

Persona o entidad que se apoya en un analista, con el fin de proponer una solución a un problema. Es quien elabora el discurso de los requisitos, los cuales encaminan la solución.

Equipo ágil

Conjunto de personas expertas en áreas de software y en el dominio del negocio, que trabajan orientadas hacia un mismo fin en pequeños ciclos de tiempo, llamados iteraciones. Las iteraciones se presentan varias fases: análisis, diseño, implementación y pruebas.

Metodología ágil

Proceso de desarrollo de software que busca construir aplicaciones de manera rápida teniendo en cuenta el interesado para la mayoría de pasos que el método contiene. Las metodologías ágiles enfatizan en las comunicaciones cara a cara en vez de la documentación.

Funcionalidad

En este dominio, se define como un conjunto de características que cumplen una tarea específica en una aplicación de software.

Iteración

Se puede definir como la implementación de una funcionalidad en una unidad de tiempo.

Prueba

Procedimiento para validar una funcionalidad de la aplicación de software. Existen varios tipos de prueba, entre ellas por unidad, por componente, de exploración y de regresión.

XUnit

Entorno utilizado para automatizar las pruebas unitarias. Actualmente se vienen desarrollando *frameworks* para su implementación.

Bug Tracking System

Herramienta utilizada para el seguimiento de bugs manejada por los equipos de desarrollo para conocer qué fallos se deben corregir y cuáles no.

La filosofía de calidad toma forma en el desarrollo de software cuando todos miembros del equipo son probadores, pues desaparece la fase de pruebas, que forma parte de los métodos tradicionales.

Es importante dejar en claro que la codificación y las pruebas constituyen un solo proceso integrado y, por ello, el tiempo empleado en codificación puede ser el mismo a emplear en las pruebas de esa aplicación de software. Es por ello por lo que la experiencia de los probadores se hace vital a la hora de definir requisitos del cliente y su estimación de codificación y pruebas, porque el nombre ágil no significa entregar productos rápidamente con la funcionalidad solicitada, sino que, además, deben ser confiables.

El diseño de pruebas, como tal, y el uso de las herramientas adecuadas se vuelven vitales para asegurar el principio de calidad. La planeación y el diseño de pruebas es importante a la hora de distribuir tiempo en *testing* particular, arreglar bugs y realizar otro tipo de pruebas como pruebas de aceptación, pruebas de regresión, pruebas por unidad, pruebas por componente y pruebas de exploración.

El factor de éxito del *testing* ágil es saber automatizar las pruebas de regresión, las cuales aseguran calidad y precisión sobre todo el comportamiento del sistema. Además, se deberían automatizar las pruebas de aceptación, que aseguran al cliente la funcionalidad que requirió en un principio.

Recordatorio de **tipos de estrategias de pruebas**:

Exploratorio

Prueba no guionada, ni repetitiva. Suele ser realizada con una misión definida (por ejemplo, probar la usabilidad o reproducir un defecto), con tiempo limitado (*timeboxed*), y se realiza de modo simultáneo la planificación, diseño y ejecución de las pruebas, en ciclos cortos (minutos) que permiten adaptar el nuevo ciclo a los resultados obtenidos en el anterior, en un proceso de aprendizaje. Puede tener soporte de herramientas, pero no puede ser automatizado. Finaliza con un reporte de la prueba en el que se describe las hipótesis, las pruebas realizadas, los resultados de las mismas, y los posibles pasos a seguir.

Prueba automatizada

Estas pruebas son construidas completamente antes de la ejecución de las mismas, por lo que no se requiere intervención humana durante la ejecución. Esto permite que se realicen frecuente y rápidamente.

Aceptación

Pruebas realizada por el usuario (o cliente o sus representantes) que definen si el producto se ajusta a las necesidades y puede ser aceptado. Según la relación entre el usuario y el equipo de desarrollo puede ser que la aceptación se refiera sólo al cumplimiento de las necesidades explicitadas o sea más amplio.

Funcional

Pruebas desde el punto de vista de la funcionalidad, y no sobre temas como confiabilidad, mantenibilidad, usabilidad, seguridad. Dado que la definición de funcionalidad puede ser ambigua (por ejemplo, se puede considerar que no tiene sentido distinguir entre requerimientos funcionales y no funcionales), en ocasiones se usa este término con el mismo sentido que prueba de sistema.

Integración

Pruebas orientadas a las interacciones entre los distintos componentes que componen el producto. Se buscan que se comuniquen correctamente tanto desde el punto de vista sintáctico como semántico.

Sistemas

Pruebas de punta a punta, considerando no sólo el producto que el equipo está desarrollando, sino también otros productos (de software o no), procesos y personas involucradas en la solución a las necesidades del usuario.

Otros tipos o estrategias: *fuzzing*, combinatoria, guiada por riesgos, *Behavior-Driven Development* (DBB), caja negra, caja blanca, pruebas de regresión, pruebas formales.

TDD - *Test-Driven Development*

Una forma de diseño y programación, que se basa en declarar el diseño en la forma de ejemplos ejecutables (pruebas) del uso del componente, antes de programar el componente.

El ciclo de TDD consiste en: escribir las pruebas, ver que fallen (no está el código correspondiente), escribir el código mínimo para lograr que la prueba pase, ver que las pruebas pasen, y luego revisar el código (del componente y de las pruebas) para mejorarlo (también llamado *refactoring*).

Esta técnica se origina en *Extreme Programming*, y tiene mucha relación con otras como *Pair Programming*, *Simple Design*, *Coding Standard*. No es una técnica de prueba en sí misma, pero genera como subproducto un conjunto de pruebas automatizadas con alta cobertura de código, lo que implica que el software desarrollado de esta manera suele tener niveles de calidad muy buenos.

BDD - *Behaviour-Driven Development*

- BDD combina las técnicas generales y los principios de TDD con las ideas del diseño basado en el dominio.
- BDD es un conjunto de mejores prácticas para escribir grandes pruebas.
- BDD puede, y debe ser, utilizado junto con TDD y métodos de prueba de unidad.
- BDD no debe probar la implementación, sino el comportamiento.

ATDD - *Acceptance Test-Driven Development*

ATDD viene de Acceptance Test-Driven Development.

Los usuarios y/o al *product owner*, desarrolladores y *tester*, analizan conjuntamente los criterios de aceptación, antes de que comience el desarrollo.

Aquí hay dos prácticas claves para estar ante un ATDD:

- Antes de implementar (resalto lo de antes de implementar) una necesidad, requisito, historia de usuario, etc., los miembros del equipo (usuarios, o *product owners*, desarrolladores, *tester*, etc.) colaboran para crear escenarios, ejemplos, de cómo se comportará dicha necesidad.
- Después, el equipo convierte esos escenarios en pruebas de aceptación automatizadas. Estas pruebas de aceptación típicamente se automatizan.

¿Qué es *testing* ágil y qué no?

¿Qué es agile <i>testing</i> ?	¿Qué no es agile <i>testing</i> y sí es QA?
1. Los test ágiles están integrados como miembros de pleno en el equipo de desarrollo y participan en la planificación, estimación y todas las actividades del equipo.	Las pruebas se realizan de manera separada o grupo con uno o más equipos de ingenieros de control de la calidad.
2. Los test ágiles trabajan mano a mano en el desarrollo y gestión del producto.	Los equipos pueden trabajar con otros grupos, pero con interacciones limitadas y bien establecidas.
3. Los requerimientos del negocio se escriben uno a la vez para dar espacio a cambios empresariales.	QA y el equipo de desarrollo reciben requerimientos de negocio por adelantado.
4. Los desarrolladores a menudo toman la iniciativa en las pruebas a nivel del código, mientras que los test ágiles se encuentran o centran a la	Mientras los desarrolladores escriben código, QA escriben el plan de prueba, y casos de pruebas para soportar los requerimientos.

automatización de pruebas de aceptación, y construcciones de pruebas de regresión.	
6. La experiencia ágil es continua y la retroalimentación se proporciona en todas las etapas.	QA sigue un ciclo de pruebas y errores, donde todo error encontrado es devuelto al desarrollador para ser corregido.
7. Los test ágiles siguen un proceso bien definido, y continuo a los defectos físicos encontrados.	Cuando se finaliza el proceso de desarrollo pasa al QA para ser revisado.
9. La calidad del software siempre está lista para ser entregada.	El proceso de corrección de pruebas se repite hasta agotar el tiempo de desarrollo.

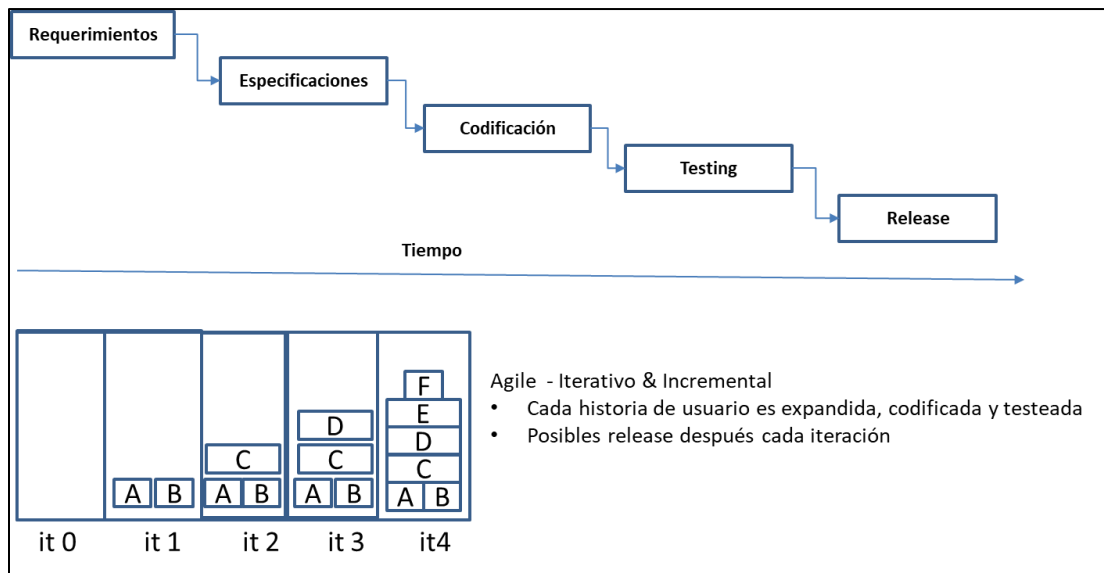
Proyectos tradicionales vs. Proyectos ágiles

Los proyectos ágiles:

- Son distintos a los "tradicionales" en que son más rápidos (iteraciones cortas, y software funcionando antes), pero igualmente tienen los mismos problemas.
- Se planifican, de a poquito, permite responder rápidamente al cambio para entregar valor.
- La documentación existe, pero es más fluida.
- Se le da más importancia al código que funciona.
- El equipo prefiere dedicar su esfuerzo en crear pruebas automatizadas que soporten al código.
- Para los probadores, el desafío más grande es que el software está siempre cambiando.

A ambos le llegan malos requerimientos.

A ambos también tienen que resolver problemas de coordinación con múltiples participantes.

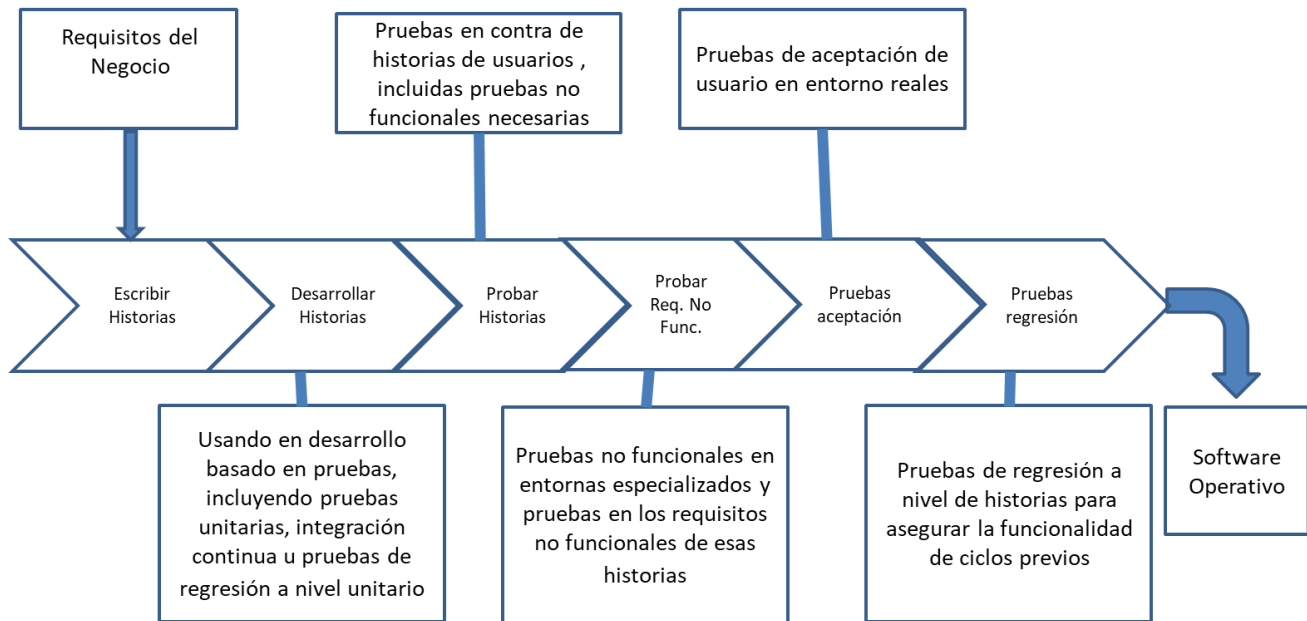


El sistema bajo prueba se transforma en un blanco móvil. Se necesitan nuevos enfoques de prueba para enfrentar este tipo de cambios, con diferentes habilidades, tácticas y herramientas.

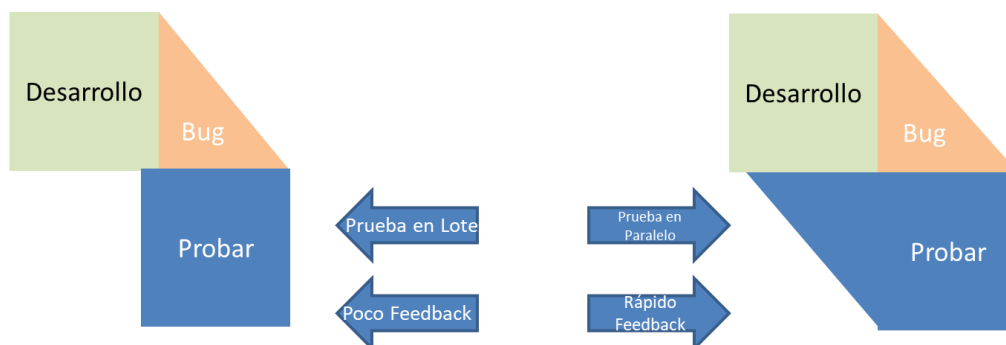
Algunos aspectos del testeo que no cambian, como ser la ejecución de las pruebas, pero muchos probadores necesitarán realizar cambios importantes a su estrategia de *testing* para brindar valor real a un proyecto de software ágil.

Proceso de ágil

El cambio de estrategia:



Pensemos en un espectro para el test de software. En un extremo tenemos un testeo manual totalmente guionado. Estamos hablando de pruebas escritas en tablas, completamente llenas de casos a testear. Esta es una punta del espectro. En el otro lado tenemos el *testing* de exploración libre, sin guiones para testear, usando sólo la experiencia y la intuición para encontrar problemas.



El *testing* exploratorio es invaluable para los proyectos ágiles. En muchos casos, el testeo exploratorio se alinea con el manifiesto ágil, respondiendo rápidamente al cambio y enfocándose en solucionar el problema. Como resultado, las habilidades y tácticas para realizar un testeo exploratorio se convierten en grandes aliados para testear un software que cambia continuamente.

Si la mayor parte de tu experiencia fue testeando proyectos que seguían un enfoque "en cascada", seguramente estás acostumbrado a pasar gran parte del tiempo diseñando los guiones de test basándote en documentos de requerimientos que indican cómo validar

características específicas. Esta es una práctica muy aceptada para testear software, pero en realidad no funciona bien en los proyectos ágiles.

Uno de los motivos por los cuales esta práctica es inapropiada para proyectos ágiles es que necesita bastante tiempo inicial para que funcione.

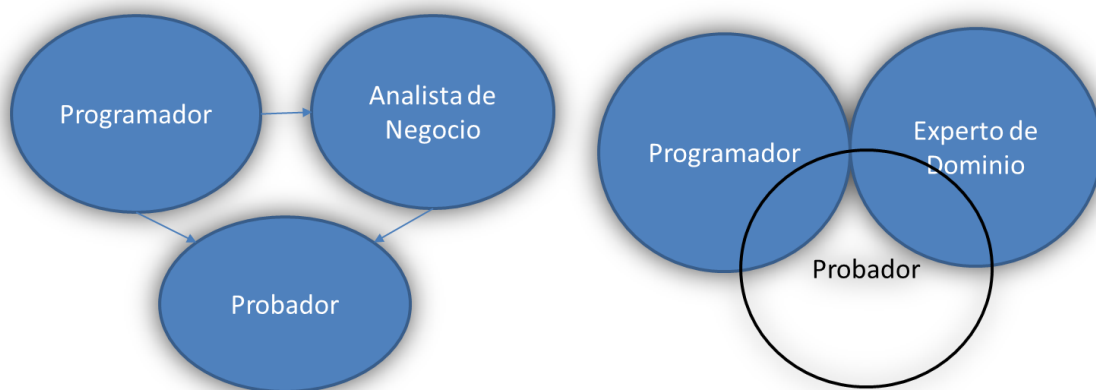
La mayoría de los proyectos ágiles tienen *sprints* de dos a cuatro semanas, y los casos de pruebas no se escriben en una noche.

En general se necesitan de tiempo que un proyecto ágil no tiene.

El test exploratorio es mucho más liviano. Tiene menos documentación, pero realizado de manera adecuada puede brindar la misma cobertura y el mismo grado de riesgo. El tema es encontrar el balance exacto en crear la estructura necesaria para realizar el seguimiento de lo realizado y poder evaluar los resultados. Una alternativa es usar el enfoque de gestión de *tests* basado en sesiones, creado por James Bach y Jon Bach.

Por otro lado, el *testing* exploratorio no necesita mucha preparación por anticipado. Es muy importante porque las metodologías ágiles hacen énfasis en el voluntarismo y la planificación constante. El voluntarismo quiere decir que, dentro del alcance de un sprint, los miembros del equipo elijen las tareas que realizarán. La planificación constante significa que las características del producto se re-priorizan al inicio de cada sprint, basándose en las nuevas necesidades acorde al mercado y la situación. El efecto es simple: no hay un plan para cada una de todas las posibles características, ni para el proyecto global. Las características pueden ir cambiando y acomodándose según cambien las necesidades. En consecuencia, se necesita poder testear cualquier cosa, en cualquier momento, con la mínima preparación posible.

Agilizando el proceso de *testing*:



Ya sin requerimientos detallados y sin un proceso de control de cambios que mantengan a estos requerimientos estáticos, la habilidad más importante es la de tomar decisiones acerca del trabajo que se hará a continuación, cómo se llevará a cabo, y cómo le daremos valor a nuestro cliente. Todo se basa en ver el software como historias que le dan valor a los usuarios, y estructurar el trabajo alrededor de estas historias (muy parecido a como los desarrolladores organizan su trabajo: historias alrededor de las cuales se organiza el trabajo).

En un proyecto con cambios continuos, es importante tomar buenas decisiones respecto a lo que se testea, y cuándo se testea. Si se prueban características que todavía no están terminadas y se generan defectos, perderemos credibilidad ante el resto del equipo (y esto es un error común que se comete cuando el software evoluciona rápidamente). Necesitamos estar siempre conscientes y en conocimiento del estado del software. Esto significa hablar con los desarrolladores, prestar atención en las reuniones diarias, y estar familiarizados y atentos a cualquier sistema que se para seguimiento del proyecto.

Una práctica que puede resultar útil es el tour de aplicación. El tour de aplicación consiste simplemente en usar la aplicación de distintas maneras para mejorar nuestro entendimiento sobre cómo funcionan las cosas, y en dónde podrían estar los riesgos. El objetivo es tan tanto encontrar defectos, sino más bien crear un modelo mental de cómo funciona el software, qué hace, y cuáles características fueron implementadas. Todos los *testers* deberían invertir algo de tiempo en hacer un tour por la aplicación en cada iteración. Esto ayuda a mantenerse en contacto con la dirección general que está tomando la aplicación.

Asegurarse de no quedar afuera

El respeto y la credibilidad son críticos en un equipo de proyectos ágiles. En estos equipos, los desarrolladores suelen tener mucha habilidad, no sólo en la codificación sino también en la creación de *tests* automáticos, especialmente si usan prácticas como Desarrollo Guiado por Test (TDD). No cometan el error de tratarlos como si no supieran de qué se trata el *testing*.

Es importante ganarse el respeto del equipo de desarrollo. Hay varias formas de lograrlo, como el juego "cinco bugs en cinco minutos". Pero en última instancia la mejor forma de ganarse el respeto es poder agregar valor al proyecto. Algunas formas que se puede lograr:

- Brindar una perspectiva nueva y fresca sobre la aplicación. Es común que los desarrolladores estén tan inmersos en la aplicación que pierdan el punto de vista del usuario real.
- Hacer un buen trabajo aislando los bugs. Cuando se encuentra un defecto, invertir algo de tiempo extra en encontrar la manera más simple de reproducirlo. Además, usar software para poder grabar la prueba (como Selenium) y adjuntar el resultado al reporte de error.
- Hablar con los desarrolladores antes de ingresar un reporte de error. A veces un único defecto puede tener varios síntomas, y en general no es útil crear un reporte por cada síntoma encontrado. El desarrollador puede ayudarnos a comprender las implicancias del defecto, y evitar así ingresar docenas de informes que en última instancia responden al mismo problema.

Los mejores equipos ágiles quieren la ayuda de *testing*. Reconocen todo el valor extra que brinda un buen *tester* que está enfocado en el proyecto. Lo importante es no quedar aislado. Esto significa que el *tester* tiene que convertirse en un consejero del equipo. Hay que averiguar qué información valora el equipo y qué información necesita (podrían no coincidir), y enfocarse en brindar un balance entre esta información.

Los equipos ágiles son rápidos, con muchos de los mismos problemas que los proyectos tradicionales (afortunadamente, estos problemas tienden a aparecer más rápido), y con algunos desafíos propios. Convertirse en un *tester* efectivo en este ambiente puede requerir cambiar la estrategia y encontrar nuevas técnicas para estructurar el trabajo y aprender más eficientemente. Y no olvidarse que una parte importante dentro del proyecto es también divertirse.

Si realmente no querés quedarte atrás, podés dejar que la diversión sea el barómetro que mida tu situación. En la mayoría de los equipos ágiles, el respeto se gana a través de la capacidad para resolver problemas, y las personas disfrutan trabajar junto a quienes respetan.